HTTP & REST Fundamentals

Topic 1: The foundation of every API

ML + UI + API + MCP Project

Learning Path

October 12, 2026

Outline

1. Why HTTP and REST?
2. Request and Response
3. Methods and Status Codes
4. JSON and Example
5. REST Principles
6. Takeaways

Why start here?

Key idea

Every API call, UI request, and MCP tool wrapper you will build rides on top of HTTP.

- **HTTP**: the protocol clients and servers use to talk.
- **REST**: a style for designing APIs around resources.
- **JSON**: the common data format in the body.

ML application

Your linear regression model will live behind a `POST /predict` endpoint: the UI sends features, the API returns a prediction.

The client–server cycle



Stateless

Each request carries everything the server needs; the server keeps no memory between calls.

Anatomy of a request and a response

Request

- **Method**: what to do
- **URL**: which resource
- **Headers**: metadata (`Content-Type`)
- **Body**: data (JSON)

Response

- **Status code**: result of the call
- **Headers**: metadata
- **Body**: returned data (JSON)

HTTP methods (CRUD)

Method	Action	Example	Safe?
`GET`	Read	`GET /models`	Yes
`POST`	Create / run	`POST /predict`	No
`PUT`	Replace	`PUT /models/1`	No
`PATCH`	Update part	`PATCH /models/1`	No
`DELETE`	Remove	`DELETE /models/1`	No

ML application

Predictions use `POST` because features travel in the body, not the URL.

Status codes

Range	Meaning	Common codes
2xx	Success	200 OK, 201 Created
3xx	Redirect	301, 304
4xx	Client error	400, 401, 404, 422
5xx	Server error	500, 503

Remember

422 = valid JSON but wrong fields (e.g. text instead of a number); 500 = your server code failed.

JSON payloads for a prediction

Request body

```
POST /predict
Content-Type: application/json

{
  "square_feet": 1850,
  "bedrooms": 3
}
```

Response body

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "predicted_price": 412500.0
}
```

Calling an API from Python

```python
import requests

url = "http://127.0.0.1:8000/predict"
payload = {"square_feet": 1850, "bedrooms": 3}

response = requests.post(url, json=payload, timeout=10)
print(response.status_code)    # 200
print(response.json())         # {'predicted_price': 412500.0}
```

REST principles at a glance

- **Resources**: nouns in URLs (`/models`), not verbs.
- **Stateless**: no session memory on the server.
- **Uniform interface**: standard methods and status codes.
- **Representations**: resources exchanged as JSON.
- **Client–server**: UI and backend evolve independently.
- **Cacheable**: `GET` responses can be reused.

Takeaways

1. HTTP is a request/response conversation between client and server.
2. Choose the method by intent: `GET` reads, `POST` sends data.
3. Status codes tell the client what happened (2xx, 4xx, 5xx).
4. JSON is the shared language between UI, API, and MCP.
5. Next topic: **API Design Principles**.